

Practical File
On

DATABASE MANAGEMENT SYSTEMS

Submitted in partial fulfilment of the requirements for
the award of the degree

Bachelor of Technology
in
Department of Computer Science and Engineering

Submitted to:

Dr. Sonu Mehla
Assistant Professor
CSE/SOET

Submitted by:

Harshit Khemani
241302081
B.Tech CSE (AI/ML)
Section - C
5th Semester

INDEX

S. No.	Name of Experiment	Date	Sign.
1	Employee Directory	19 Aug 2026	
2	CREATE & ALTER TABLE	26 Aug 2026	
3	SQL Joins	02 Sep 2026	
4			
5			
6			
7			
8			
9			
10			
11			
12			
13			
14			
15			

Ex – 1

Employee Directory

1. Aim

To design a relational schema for an employee directory with departments and employees, enforce primary-key and foreign-key constraints, create a reporting view, and verify the design with sample queries in SQLite.

2. Theory

2.1 Tables and keys

A departments table stores each organisational unit. An employees table stores people; department_id is a foreign key to departments, and manager_id is a self-referencing foreign key for the reporting hierarchy.

2.2 Views

A view (employee_directory) joins employees to their department and manager names so reporting queries do not repeat join logic.

3. Schema

Table	Key columns	Role
departments	department_id PK	Organisational units
employees	employee_id PK; department_id FK	Staff records
employees	manager_id FK → employees	Reporting chain
employee_directory	VIEW	Flattened directory for queries

4. Procedure

1. Create departments and employees with constraints and indexes.
2. Create the employee_directory view.
3. Insert six departments and twenty-six employees with a three-tier org chart.
4. Run aggregate queries on the view (counts by department).

5. Source Code

File: 19-08-2026/schema.sql

```
-- =====
-- Employee Directory – schema
-- SQLite dialect. Runs as-is in the in-browser SQL Compiler
-- (index.html), and in the sqlite3 CLI / DB Browser for SQLite.
-- =====

PRAGMA foreign_keys = ON;

DROP VIEW IF EXISTS employee_directory;
DROP TABLE IF EXISTS employees;
DROP TABLE IF EXISTS departments;

-- -----
-- departments
-- -----
CREATE TABLE departments (
  department_id INTEGER PRIMARY KEY AUTOINCREMENT,
  name          TEXT NOT NULL UNIQUE,
```

```

        description    TEXT,
        color_hex      TEXT NOT NULL DEFAULT '#1CB0F6'
    );

-- -----
-- employees
-- -----
CREATE TABLE employees (
    employee_id        INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name         TEXT NOT NULL,
    last_name          TEXT NOT NULL,
    email              TEXT NOT NULL UNIQUE,
    job_title           TEXT NOT NULL,
    department_id       INTEGER NOT NULL REFERENCES departments (department_id) ON DELETE RESTRICT,
    manager_id         INTEGER REFERENCES employees (employee_id) ON DELETE SET NULL,
    salary              NUMERIC(10,2) NOT NULL CHECK (salary > 0),
    hire_date           TEXT NOT NULL,
    employment_status   TEXT NOT NULL DEFAULT 'active'
                        CHECK (employment_status IN ('active', 'remote', 'on_leave')),
    avatar_seed         TEXT NOT NULL
);

CREATE INDEX idx_employees_department ON employees (department_id);
CREATE INDEX idx_employees_manager   ON employees (manager_id);

-- -----
-- employee_directory – flattened view joining department + manager.
-- Convenient for reporting queries and reused by the frontend.
-- -----
CREATE VIEW employee_directory AS
SELECT
    e.employee_id,
    e.first_name || ' ' || e.last_name AS full_name,
    e.email,
    e.job_title,
    d.name                               AS department_name,
    d.color_hex                         AS department_color,
    m.first_name || ' ' || m.last_name AS manager_name,
    e.salary,
    e.hire_date,
    e.employment_status,
    e.avatar_seed
FROM employees e
JOIN departments d ON d.department_id = e.department_id
LEFT JOIN employees m ON m.employee_id = e.manager_id;

```

6. Output

```

SQL> SELECT COUNT(*) AS dept_count FROM departments;
dept_count
6

```

```

SQL> SELECT COUNT(*) AS emp_count FROM employees;
emp_count
26

```

```

SQL> SELECT department_name, COUNT(*) AS headcount
FROM employee_directory
GROUP BY department_name
ORDER BY headcount DESC
LIMIT 5;
department_name  headcount
Engineering      8
Design           5
Marketing         4
Sales            4
People Ops       3

```

7. Results

Six departments and twenty-six employees load successfully. Foreign keys prevent orphan department_id values. The view returns each employee with department and manager names for browsing and SQL practice.

8. Conclusion

Normalised tables plus a view provide a maintainable employee directory. Primary and foreign keys preserve referential integrity; the view simplifies read-only reporting queries used in the web compiler.

Ex – 2

CREATE & ALTER TABLE

1. Aim

To create an employees table with attributes empID, empName, department and salary, insert sample records, and then rename the column department to faculty using ALTER TABLE.

2. Theory

2.1 CREATE TABLE

A relational table is defined by a name, a list of attributes, and constraints that protect the data. PRIMARY KEY makes empID unique and not null. NOT NULL rejects missing names and faculty values. CHECK (salary > 0) rejects non-positive salaries. NUMERIC(10, 2) stores pay as a decimal with two fractional digits.

2.2 ALTER TABLE RENAME COLUMN

In SQLite and MySQL 8+, ALTER TABLE ... RENAME COLUMN changes a column's name without rewriting the stored values. After RENAME COLUMN department TO faculty, every row still holds the same school name; only the heading of that column becomes faculty. This is a schema change, not an UPDATE. Microsoft SQL Server does not accept that syntax; the equivalent is EXEC sp_rename 'employees.department', 'faculty', 'COLUMN' (see employees.sqlserver.sql).

3. Schema

The table is created with four attributes. After the rename, the third attribute is faculty instead of department.

Attribute	Type	Constraint	Role
empID	INTEGER	PRIMARY KEY	Employee identifier
empName	TEXT	NOT NULL	Employee name
department → faculty	TEXT	NOT NULL	School / faculty name
salary	NUMERIC(10,2)	NOT NULL, CHECK > 0	Monthly salary

Sample faculty values include **School of Engineering and Technology** (the faculty named on this report's cover). The cover also lists **Department of CSE** as the student's home department; that heading is letterhead, not a fifth column in the table.

4. Procedure

1. Drop employees if it already exists, so the script can be re-run.
2. Create the table with empID, empName, department and salary.
3. Insert six sample employees.
4. Display all rows and the column list with PRAGMA table_info.
5. Rename department to faculty.
6. Display all rows and the column list again to confirm that only the name changed.

5. Source Code

```
-- =====
-- DBMS Lab · 26-08-2026
-- Employees table: CREATE, seed, then RENAME department → faculty
```

```
-- SQLite (sql.js compiler / DB Browser / sqlite3)
-- For the class SQL Server, run employees.sqlserver.sql instead.
-- =====

DROP TABLE IF EXISTS employees;

-- -----
-- 1. Create the employees table
-- -----
CREATE TABLE employees (
    empID      INTEGER PRIMARY KEY,
    empName    TEXT NOT NULL,
    department TEXT NOT NULL,
    salary     NUMERIC(10, 2) NOT NULL CHECK (salary > 0)
);

-- -----
-- 2. Insert sample rows (department holds the faculty/school name)
-- -----
INSERT INTO employees (empID, empName, department, salary) VALUES
    (1, 'Ananya Sharma', 'School of Engineering and Technology', 92000.00),
    (2, 'Rohan Mehta', 'School of Management', 78000.00),
    (3, 'Priya Nair', 'School of Law', 85000.00),
    (4, 'Vikram Singh', 'School of Agricultural Sciences', 76000.00),
    (5, 'Fatima Khan', 'School of Engineering and Technology', 88000.00),
    (6, 'Arjun Patel', 'School of Medical Sciences', 95000.00);

-- -----
-- 3. Verify the table BEFORE the rename
-- -----
SELECT * FROM employees;
PRAGMA table_info(employees);

-- -----
-- 4. Rename the column department → faculty
-- Row values are unchanged; only the column name changes.
-- -----
ALTER TABLE employees RENAME COLUMN department TO faculty;

-- -----
-- 5. Verify the table AFTER the rename
-- -----
SELECT * FROM employees;
PRAGMA table_info(employees);
```

6. Output

The script was executed in SQLite. PRAGMA table_info columns are cid (column index), name, type, notnull, dflt_value and pk.

```
SQL> DROP TABLE IF EXISTS employees;
-- OK

SQL> CREATE TABLE employees (
    empID      INTEGER PRIMARY KEY,
    empName    TEXT NOT NULL,
    department TEXT NOT NULL,
    salary     NUMERIC(10, 2) NOT NULL CHECK (salary > 0)
);
-- OK

SQL> INSERT INTO employees (empID, empName, department, salary) VALUES
    (1, 'Ananya Sharma', 'School of Engineering and Technology', 92000.00),
    (2, 'Rohan Mehta', 'School of Management', 78000.00),
    (3, 'Priya Nair', 'School of Law', 85000.00),
    (4, 'Vikram Singh', 'School of Agricultural Sciences', 76000.00),
    (5, 'Fatima Khan', 'School of Engineering and Technology', 88000.00),
    (6, 'Arjun Patel', 'School of Medical Sciences', 95000.00);
-- 6 row(s) inserted

SQL> SELECT * FROM employees;
empID empName      department                                salary
-----
1     Ananya Sharma School of Engineering and Technology      92000
2     Rohan Mehta   School of Management                     78000
3     Priya Nair    School of Law                           85000
4     Vikram Singh  School of Agricultural Sciences          76000
5     Fatima Khan   School of Engineering and Technology     88000
6     Arjun Patel   School of Medical Sciences              95000

SQL> PRAGMA table_info(employees);
cid  name      type      notnull  dflt_value  pk
----
0    empID    INTEGER   0        0            1
```

```

1  empName  TEXT          1          0
2  department TEXT        1          0
3  salary   NUMERIC(10, 2) 1          0

SQL> ALTER TABLE employees RENAME COLUMN department TO faculty;
-- OK

SQL> SELECT * FROM employees;
empID  empName      faculty                                     salary
-----
1      Ananya Sharma School of Engineering and Technology 92000
2      Rohan Mehta   School of Management                78000
3      Priya Nair    School of Law                       85000
4      Vikram Singh  School of Agricultural Sciences      76000
5      Fatima Khan   School of Engineering and Technology 88000
6      Arjun Patel   School of Medical Sciences           95000

SQL> PRAGMA table_info(employees);
cid  name      type      notnull  dflt_value  pk
-----
0    empID     INTEGER   0        1           1
1    empName  TEXT      1        0           0
2    faculty  TEXT      1        0           0
3    salary   NUMERIC(10, 2) 1        0           0

```

7. Results

Stage	Third column	Row count	Values changed?
Before ALTER	department	6	No — original schema
After ALTER	faculty	6	No — names only

All six employees remain. School names such as School of Engineering and Technology are unchanged. The schema listing is the proof: `cid = 2` is named `department` before the statement and `faculty` afterwards.

8. Conclusion

An employees relation can be created with a primary key, not-null attributes and a salary check, then adjusted with `ALTER TABLE ... RENAME COLUMN`. Renaming `department` to `faculty` updates the schema only. The stored values do not change, which is the correct way to relabel an attribute without rewriting the table.

Ex – 3

SQL Joins

1. Aim

To combine rows from related tables using CROSS JOIN, NATURAL JOIN, INNER JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, and SELF JOIN; to compare how each join type treats unmatched rows; and to understand when to use each one.

2. Theory

2.1 CROSS JOIN

A CROSS JOIN returns the Cartesian product of two tables: every row from the first table is paired with every row from the second. There is no join condition. With 6 departments and 7 employees the result has $6 \times 7 = 42$ rows. Cross joins are rare in everyday queries but useful for generating combinations (e.g. all size–colour pairs) or as a building block for other joins.

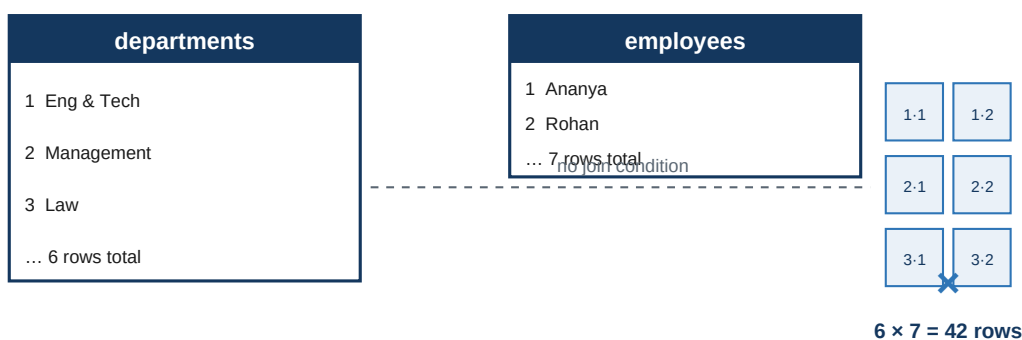


Figure 1 — CROSS JOIN pairs every department row with every employee row (42 combinations).

2.2 NATURAL JOIN

A NATURAL JOIN automatically matches rows on every column that appears in both tables with the same name. Here, both tables share deptID, so the condition is implicit. Only rows with a matching deptID in both tables are returned — the same six rows as INNER JOIN when deptID is the sole common column. (SQL Server does not support NATURAL JOIN; use an explicit INNER JOIN instead.)

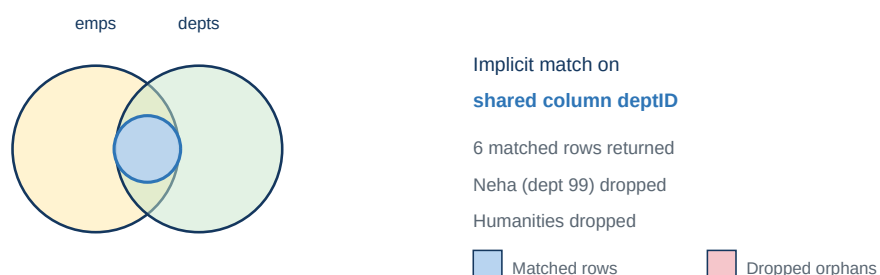


Figure 2 — NATURAL JOIN keeps only rows where deptID matches in both tables.

2.3 INNER JOIN

An **INNER JOIN** returns only rows where the explicit join condition is satisfied. Unmatched rows from either table are discarded. It is the most common join: use it when you only want records that have a partner on both sides.

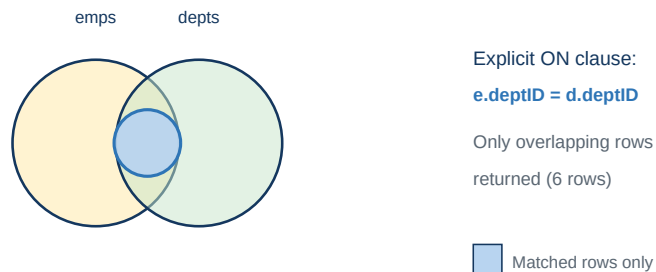
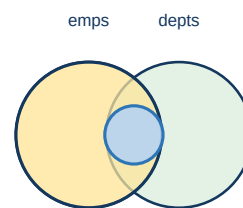


Figure 3 — INNER JOIN returns only the overlapping region (6 matched rows).

2.4 LEFT OUTER JOIN

A **LEFT OUTER JOIN** keeps every row from the left table. If no match exists in the right table, the right-hand columns are filled with **NULL**. Employee Neha Gupta (deptID 99) has no matching department and therefore appears with a null department name.

employees (left table)	
Ananya	dept=1
Rohan	dept=2
Priya	dept=3
Neha	dept=99 ← no match



All left rows kept — right columns NULL when unmatched

Figure 4 — LEFT OUTER JOIN preserves Neha (deptID 99) even though no department matches.

2.5 RIGHT OUTER JOIN

A `RIGHT OUTER JOIN` keeps every row from the right table. Unmatched left-hand columns become `NULL`. School of Humanities (deptID 6) has no employees and therefore appears with null employee fields. A `RIGHT JOIN` can always be rewritten as a `LEFT JOIN` by swapping the table order.

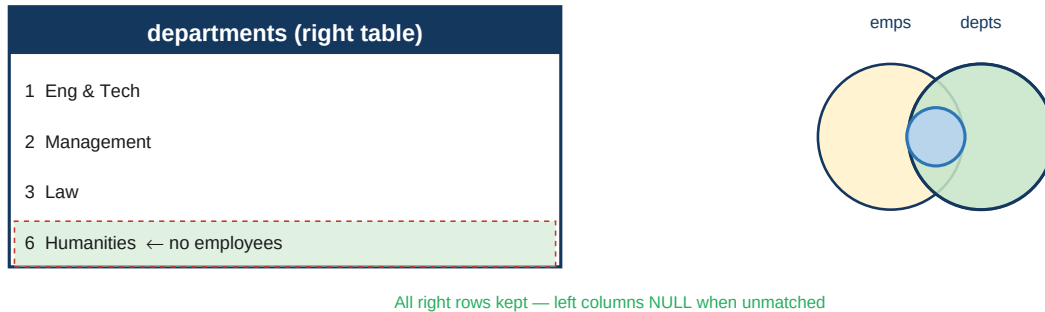


Figure 5 — `RIGHT OUTER JOIN` preserves Humanities (deptID 6) even though no employee belongs to it.

2.6 SELF JOIN

A `SELF JOIN` joins a table to itself using two different aliases. It is not a separate SQL keyword — it is an `INNER` or `LEFT JOIN` where both sides are the same table. Here, employees `e` is joined to employees `m` on `e.managerID = m.empID` to list each employee alongside their manager's name.

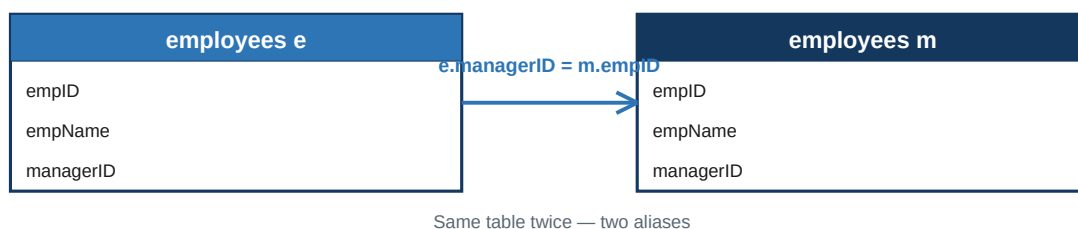


Figure 6 — `SELF JOIN`: the same table appears twice with different aliases.

2.7 Differences between all join types

Join type	Join condition?	Unmatched rows	Typical use
CROSS JOIN	None (Cartesian product)	N/A — all combinations	Generate all pairs
NATURAL JOIN	Implicit (same column names)	Dropped from both sides	Quick join on shared key
INNER JOIN	Explicit ON clause	Dropped from both sides	Only matching records
LEFT OUTER JOIN	Explicit ON clause	Left kept; right NULL	Keep all from left table
RIGHT OUTER JOIN	Explicit ON clause	Right kept; left NULL	Keep all from right table
SELF JOIN	Explicit ON (same table)	Depends on INNER/LEFT used	Hierarchies, comparisons

Key distinctions: `CROSS JOIN` multiplies rows with no filter. `INNER` and `NATURAL` return only matches. `LEFT` and `RIGHT OUTER` preserve one side's orphans. `SELF JOIN` is a technique (same table, two aliases), not a separate result category — it uses inner or outer join semantics on a single table.

3. Schema

Two tables are linked by deptID. employees also references itself through managerID for the self join (see Figure 6).

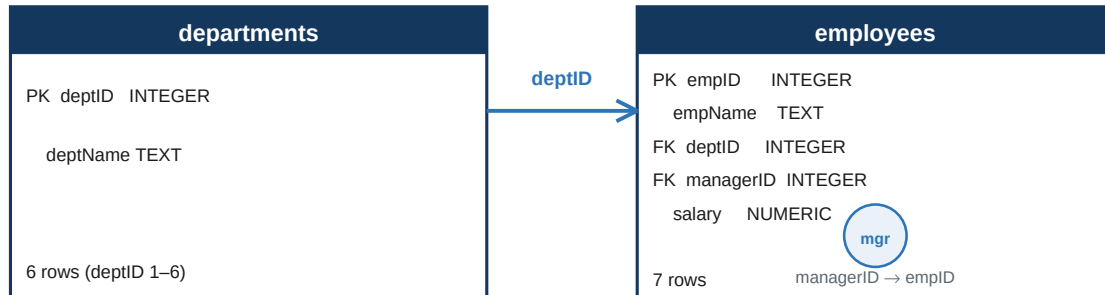


Figure 7 — Schema: employees.deptID references departments.deptID; managerID is a self-reference.

Table	Attribute	Type	Role
departments	deptID	INTEGER / INT	Primary key
departments	deptName	TEXT / VARCHAR	School name
employees	empID	INTEGER / INT	Primary key
employees	empName	TEXT / VARCHAR	Employee name
employees	deptID	INTEGER / INT	FK → departments
employees	managerID	INTEGER / INT	FK → employees (self)
employees	salary	NUMERIC(10,2)	Monthly salary

4. Procedure

1. Create departments and employees (with managerID for self join).
2. Insert six departments and seven employees, including deliberate mismatches for outer joins.
3. Display both base tables.
4. Run CROSS JOIN — expect 42 rows (count shown; sample limited to 8).
5. Run NATURAL JOIN and INNER JOIN — expect six matched rows.
6. Run LEFT OUTER JOIN — expect seven rows (one employee without a department).
7. Run RIGHT OUTER JOIN — expect seven rows (one department without employees).
8. Run SELF JOIN — expect seven rows listing each employee and their manager.

5. Source Code

```
-- =====
-- DBMS Lab · 02-09-2026
-- SQL Joins: CROSS, NATURAL, INNER, OUTER (LEFT/RIGHT), SELF
-- SQLite (sql.js compiler / DB Browser / sqlite3)
-- For the class SQL Server, run joins.sqlserver.sql instead.
-- =====

DROP TABLE IF EXISTS employees;
DROP TABLE IF EXISTS departments;
```

```

-----
-- 1. Create related tables
--   • deptID links employees → departments
--   • managerID links employees → employees (for SELF JOIN)
-----
CREATE TABLE departments (
    deptID    INTEGER PRIMARY KEY,
    deptName  TEXT NOT NULL
);

CREATE TABLE employees (
    empID     INTEGER PRIMARY KEY,
    empName   TEXT NOT NULL,
    deptID    INTEGER NOT NULL,
    managerID INTEGER,
    salary    NUMERIC(10, 2) NOT NULL CHECK (salary > 0)
);

-----
-- 2. Insert sample rows
--   • deptID 6 (Humanities) has no employees → RIGHT OUTER JOIN
--   • empID 7 points to deptID 99 (missing) → LEFT OUTER JOIN
--   • managerID links form a simple reporting chain → SELF JOIN
-----
INSERT INTO departments (deptID, deptName) VALUES
(1, 'School of Engineering and Technology'),
(2, 'School of Management'),
(3, 'School of Law'),
(4, 'School of Agricultural Sciences'),
(5, 'School of Medical Sciences'),
(6, 'School of Humanities');

INSERT INTO employees (empID, empName, deptID, managerID, salary) VALUES
(1, 'Ananya Sharma', 1, NULL, 92000.00),
(2, 'Rohan Mehta', 2, 1, 78000.00),
(3, 'Priya Nair', 3, 1, 85000.00),
(4, 'Vikram Singh', 4, 2, 76000.00),
(5, 'Fatima Khan', 1, 1, 88000.00),
(6, 'Arjun Patel', 5, 3, 95000.00),
(7, 'Neha Gupta', 99, 2, 54000.00);

-----
-- 3. Base tables (reference)
-----
SELECT * FROM departments;
SELECT * FROM employees;

-----
-- 4. CROSS JOIN
--   Cartesian product: every row of table A paired with every
--   row of table B. No join condition. Here: 6 × 7 = 42 rows.
-----
SELECT COUNT(*) AS cross_join_row_count
FROM departments
CROSS JOIN employees;

SELECT d.deptName, e.empName
FROM departments AS d
CROSS JOIN employees AS e
ORDER BY d.deptID, e.empID
LIMIT 8;

-----
-- 5. NATURAL JOIN
--   Matches on every column with the same name (here: deptID).
--   Only rows with a matching deptID in BOTH tables appear.
-----
SELECT empID, empName, deptName, salary
FROM employees
NATURAL JOIN departments;

-----
-- 6. INNER JOIN
--   Explicit join condition; same result as NATURAL JOIN here
--   because deptID is the only common column.
-----
SELECT e.empID, e.empName, d.deptName, e.salary
FROM employees AS e
INNER JOIN departments AS d ON e.deptID = d.deptID;

-----
-- 7a. LEFT OUTER JOIN
--   All rows from the LEFT table (employees); unmatched dept
--   columns are NULL (empID 7 → deptID 99 has no department).
-----
SELECT e.empID, e.empName, e.deptID, d.deptName, e.salary
FROM employees AS e
LEFT OUTER JOIN departments AS d ON e.deptID = d.deptID;

```

```

-- -----
-- 7b. RIGHT OUTER JOIN
-- All rows from the RIGHT table (departments); unmatched
-- employee columns are NULL (deptID 6 has no employees).
-- Requires SQLite 3.39+; see joins.sqlserver.sql for T-SQL.
-- -----
SELECT e.empID, e.empName, d.deptID, d.deptName, e.salary
FROM employees AS e
RIGHT OUTER JOIN departments AS d ON e.deptID = d.deptID;

-- -----
-- 8. SELF JOIN
-- A table joined to itself using different aliases.
-- Here: each employee matched to their manager (also in
-- employees). Ananya has no manager → manager name is NULL.
-- -----
SELECT
    e.empName AS employee,
    m.empName AS manager
FROM employees AS e
LEFT JOIN employees AS m ON e.managerID = m.empID
ORDER BY e.empID;

```

6. Output

The script was executed in SQLite. Each query's result set is shown below in execution order.

```

SQL> DROP TABLE IF EXISTS employees;
-- OK

SQL> DROP TABLE IF EXISTS departments;
-- OK

SQL> CREATE TABLE departments (
    deptID INTEGER PRIMARY KEY,
    deptName TEXT NOT NULL
);
-- OK

SQL> CREATE TABLE employees (
    empID INTEGER PRIMARY KEY,
    empName TEXT NOT NULL,
    deptID INTEGER NOT NULL,
    managerID INTEGER,
    salary NUMERIC(10, 2) NOT NULL CHECK (salary > 0)
);
-- OK

SQL> INSERT INTO departments (deptID, deptName) VALUES
    (1, 'School of Engineering and Technology'),
    (2, 'School of Management'),
    (3, 'School of Law'),
    (4, 'School of Agricultural Sciences'),
    (5, 'School of Medical Sciences'),
    (6, 'School of Humanities');
-- 6 row(s) inserted

SQL> INSERT INTO employees (empID, empName, deptID, managerID, salary) VALUES
    (1, 'Ananya Sharma', 1, NULL, 92000.00),
    (2, 'Rohan Mehta', 2, 1, 78000.00),
    (3, 'Priya Nair', 3, 1, 85000.00),
    (4, 'Vikram Singh', 4, 2, 76000.00),
    (5, 'Fatima Khan', 1, 1, 88000.00),
    (6, 'Arjun Patel', 5, 3, 95000.00),
    (7, 'Neha Gupta', 99, 2, 54000.00);
-- 13 row(s) inserted

SQL> SELECT * FROM departments;
deptID deptName
-----
1 School of Engineering and Technology
2 School of Management
3 School of Law
4 School of Agricultural Sciences
5 School of Medical Sciences
6 School of Humanities

SQL> SELECT * FROM employees;
empID empName deptID managerID salary
-----
1 Ananya Sharma 1 92000
2 Rohan Mehta 2 1 78000
3 Priya Nair 3 1 85000
4 Vikram Singh 4 2 76000
5 Fatima Khan 1 1 88000
6 Arjun Patel 5 3 95000
7 Neha Gupta 99 2 54000

```

```
SQL> SELECT COUNT(*) AS cross_join_row_count
      FROM departments
      CROSS JOIN employees;
cross_join_row_count
-----
42
```

```
SQL> SELECT d.deptName, e.empName
      FROM departments AS d
      CROSS JOIN employees AS e
      ORDER BY d.deptID, e.empID
      LIMIT 8;
```

deptName	empName
School of Engineering and Technology	Ananya Sharma
School of Engineering and Technology	Rohan Mehta
School of Engineering and Technology	Priya Nair
School of Engineering and Technology	Vikram Singh
School of Engineering and Technology	Fatima Khan
School of Engineering and Technology	Arjun Patel
School of Engineering and Technology	Neha Gupta
School of Management	Ananya Sharma

```
SQL> SELECT empID, empName, deptName, salary
      FROM employees
      NATURAL JOIN departments;
```

empID	empName	deptName	salary
1	Ananya Sharma	School of Engineering and Technology	92000
2	Rohan Mehta	School of Management	78000
3	Priya Nair	School of Law	85000
4	Vikram Singh	School of Agricultural Sciences	76000
5	Fatima Khan	School of Engineering and Technology	88000
6	Arjun Patel	School of Medical Sciences	95000

```
SQL> SELECT e.empID, e.empName, d.deptName, e.salary
      FROM employees AS e
      INNER JOIN departments AS d ON e.deptID = d.deptID;
```

empID	empName	deptName	salary
1	Ananya Sharma	School of Engineering and Technology	92000
2	Rohan Mehta	School of Management	78000
3	Priya Nair	School of Law	85000
4	Vikram Singh	School of Agricultural Sciences	76000
5	Fatima Khan	School of Engineering and Technology	88000
6	Arjun Patel	School of Medical Sciences	95000

```
SQL> SELECT e.empID, e.empName, e.deptID, d.deptName, e.salary
      FROM employees AS e
      LEFT OUTER JOIN departments AS d ON e.deptID = d.deptID;
```

empID	empName	deptID	deptName	salary
1	Ananya Sharma	1	School of Engineering and Technology	92000
2	Rohan Mehta	2	School of Management	78000
3	Priya Nair	3	School of Law	85000
4	Vikram Singh	4	School of Agricultural Sciences	76000
5	Fatima Khan	1	School of Engineering and Technology	88000
6	Arjun Patel	5	School of Medical Sciences	95000
7	Neha Gupta	99		54000

```
SQL> SELECT e.empID, e.empName, d.deptID, d.deptName, e.salary
      FROM employees AS e
      RIGHT OUTER JOIN departments AS d ON e.deptID = d.deptID;
```

empID	empName	deptID	deptName	salary
1	Ananya Sharma	1	School of Engineering and Technology	92000
2	Rohan Mehta	2	School of Management	78000
3	Priya Nair	3	School of Law	85000
4	Vikram Singh	4	School of Agricultural Sciences	76000
5	Fatima Khan	1	School of Engineering and Technology	88000
6	Arjun Patel	5	School of Medical Sciences	95000
		6	School of Humanities	

```
SQL> SELECT
      e.empName AS employee,
      m.empName AS manager
      FROM employees AS e
      LEFT JOIN employees AS m ON e.managerID = m.empID
      ORDER BY e.empID;
```

employee	manager
Ananya Sharma	
Rohan Mehta	Ananya Sharma
Priya Nair	Ananya Sharma
Vikram Singh	Rohan Mehta
Fatima Khan	Ananya Sharma
Arjun Patel	Priya Nair
Neha Gupta	Rohan Mehta

7. Results

Row counts returned by each join on this dataset:

Join type	Rows returned	Unmatched handling
CROSS JOIN	42	No condition — every dept × every employee
NATURAL JOIN	6	Drops non-matching rows from both sides
INNER JOIN	6	Same as NATURAL JOIN here
LEFT OUTER JOIN	7	Keeps Neha Gupta (deptID 99); deptName is NULL
RIGHT OUTER JOIN	7	Keeps Humanities (deptID 6); employee cols are NULL
SELF JOIN	7	All employees; Ananya has NULL manager (top of chain)

CROSS JOIN produces the largest result because it ignores relationships entirely. INNER JOIN and NATURAL JOIN return only the six employees whose deptID exists in departments. Outer joins add the orphan employee (left) or orphan department (right). SELF JOIN does not change row count here because a LEFT JOIN keeps every employee and simply leaves the manager column null for Ananya Sharma, who has no manager.

8. Conclusion

SQL joins answer different questions about related data. CROSS JOIN asks “every combination?” INNER JOIN asks “only matches?” Outer joins ask “keep everyone from this side even without a match?” SELF JOIN asks “how do rows in this table relate to other rows in the same table?” Choosing the right join depends on whether you need all combinations, only matches, or orphan rows from a specific side.